

CSCI 2800 — Computer Architecture and Operating Systems (CAOS)
Homework 1 (document version 1.1)
User I/O and Dynamic Memory Allocation

- This homework is due in Submittity by 10:00PM Monday, September 14, 2026
- You can use at most **two late days** on this assignment
- This homework is to be done individually, so **please do not share your code with others**
- Please do **not** use AI to assist you on this homework
- Place all of your code in `hw1.c` for submission; no other files may be included
- You **must** use C for this assignment, and all submitted code **must** successfully compile via `gcc` with no warning messages when the `-Wall` (i.e., warn all) compiler option is used; we will also use `-Werror`, which will treat all warnings as critical errors
- All submitted code **must** successfully compile and run on Submittity, which currently uses Ubuntu v22.04.5 LTS and `gcc` version 11.4.0 (`Ubuntu 11.4.0-1ubuntu1~22.04.3`)
- This assignment will be entirely auto-graded by Submittity; you will have **eight** penalty-free submissions, after which points will slowly be deducted, e.g., -1 on submission #9, etc.
- You will have at least **three** days before the due date to start submitting your code to Submittity; if auto-grading is not available three days before the due date, the due date will be 10:00PM three days after auto-grading becomes available

Hints and reminders

Do **not** rely on program output to confirm whether your code is correct. Consistently allocate **exactly** the number of bytes you need regardless of whether you use static or dynamic memory allocation. Further, deallocate dynamically allocated memory via `free()` at the earliest possible point in your code.

Make use of `valgrind` (or `drmemory` or other dynamic memory checkers) or the “valgrind unlimited” gradeable in Submittity to check for errors with dynamic memory allocation and dynamic memory usage.

Always read (and re-read!) the `man` pages for library functions (section 3), system calls (section 2), etc., especially if you do not recognize a function you are required to use.

To understand how `man` pages are organized, check out the `man` page for `man` itself (via `man man`).

Homework specifications

In this first homework, you will use C to implement a cache of floating-point values, which will be populated with values entered by the user.

Your cache will be a dynamically allocated hash table of a given fixed size that stores up to three floating-point values at each cache entry. If a cache entry is full (i.e., three cached values already exist at that given key), remove the oldest of the three values; this approach is called Least Recently Used (LRU). Further, if a “cache hit” occurs, reorder the cached values oldest to newest, if necessary.

Your hash table is a two-layer structure in which the first layer consists of a one-dimensional array of `float *` pointers. These pointers should all initially be set to `NULL`, indicating that no values are stored.

As values are read in from the user, these pointers are then set to point to a dynamically allocated array of values that contains **exactly** the number of cached values. This forms the second-layer of the two-layer structure.

A good example to use as reference is `dynamic-mem-two-layer-structure.c`, as well as other examples involving dynamic memory allocation.

Note that you are allowed to use square brackets '[' and ']' on this homework, though you should start getting in the habit of using pointer arithmetic instead. In future assignments, you will not be allowed to use square brackets.

Command-line arguments and memory allocation

The first and only command-line argument specifies the size of the cache, which is the size of the dynamically allocated `float *` array that you must create. Use `calloc()` to create this array of “placeholder” pointers. But first, use `atoi()` to convert the command-line argument from a string to an integer. The cache size must be a positive integer; if not, report an error (see page 4 below).

Next, have your program prompt the user to input as many numbers as they would like, ending their input with `CTRL-D`. The prompt is displayed only once, but your program must read all floating-point values, if any, on standard input (`stdin`) and process each value in the order given.

Use `scanf()` with `"%f"` to read each floating-point value. Check the return value of `scanf()` to determine if a valid value was read in. If not, based on this return value, either call `fgetc()` to skip invalid characters or stop reading altogether if the user typed `CTRL-D`, which indicates end-of-file (EOF).

Back to the cache, remember that your cache is initially empty, meaning it is an array of `NULL` pointers. Storing each valid floating-point value therefore also requires dynamic memory allocation. For this, use `calloc()` if the cache array slot is empty; otherwise, to extend the array to store a second or third value, use `realloc()` to expand the size of the allocation.

Do **not** use `malloc()` or `memset()` in your code. Submittly will ignore these calls by removing any lines of source code that contain the string `"malloc"` or `"memset"` (including within a comment).

An example cache of size 17 is shown below.

```

                                float*
                                +-----+
cache ==> [0] | NULL |
                                +-----+
                                [1] | NULL |      [0]      [1]      [2]
                                +-----+ +-----+-----+
                                [2] |<addr>==>| 206.787 | 342.803 | 546.91 |
                                +-----+ +-----+-----+
                                .      .
                                .      .
                                .      .      [0]      [1]
                                +-----+ +-----+-----+
                                [16] |<addr>==>| 135.9998 | 101.0 |
                                +-----+ +-----+-----+
```

How do you “hash” each number?

After each successful call to `scanf()`, you must determine the hash key or cache array index for the given floating-point value. To properly “hash” each value, apply the “mod” operator to determine the remainder after dividing the integer part of the value by the cache array size. As an example, if the cache array size was 17, the value 342.803 would have a cache array index calculated as the remainder of 342/17 or 2. The value 342.803 would then be saved in the cache at index 2. For negative numbers, simply ignore the sign, e.g., if the cache array size was 17, the value -451.154 would have a cache array index of 9.

Required output

When you execute your program, you must display a line of output for each floating-point value on the input. For each value, display the cache array index and whether you called `calloc()` or `realloc()` to create or extend the array, or if the cache entry was full and you shifted existing values (i.e., `shift`), which requires you to remove the oldest value—or if you encountered a “cache hit” and did nothing (i.e., `nop`). Note that for a cache hit, you might need to reorder the existing values based on the LRU algorithm (i.e., `reorder`).

Below is an example that illustrates all of the cases above. It also shows the output format you must follow; all floating-point values must be printed with three digits after the decimal point. Note that input with a `>` prefix represents user input.

```
bash$ ./a.out 17
Enter floating-point values below (CTRL-D to end).
>123.456 456.789 789 1234.5678 5678.9012 9012.3456 abcd
Value 123.456 hashes to index 4 (calloc)
Value 456.789 hashes to index 14 (calloc)
Value 789.000 hashes to index 7 (calloc)
Value 1234.568 hashes to index 10 (calloc)
Value 5678.901 hashes to index 0 (calloc)
Value 9012.346 hashes to index 2 (calloc)
>efgh 123 17.7 34.8 51.999 333.33333 4444.4444
Value 123.000 hashes to index 4 (realloc)
Value 17.700 hashes to index 0 (realloc)
Value 34.800 hashes to index 0 (realloc)
Value 51.999 hashes to index 0 (shift)
Value 333.333 hashes to index 10 (realloc)
Value 4444.444 hashes to index 7 (realloc)
><CTRL-D>
```

When you have finished processing all of the input, display the contents of the cache by writing a line of output for each non-empty entry in the cache. For cache entries containing more than one value, write the values ordered oldest to newest. Use the following format:

```
[0] ==> 17.700, 34.800, 51.999
[2] ==> 9012.346
[4] ==> 123.456, 123.000
[7] ==> 789.000, 4444.444
[10] ==> 1234.568, 333.333
[14] ==> 456.789
```

For the `reorder` case, if the user entered 1234.5678 as one more value, the output above would be extended to contain the following last line:

```
Value 1234.568 hashes to index 10 (reorder)
```

And here, the final cache output would show entry [10] as follows:

```
[10] ==> 333.333, 1234.568
```

Error handling

If improper command-line arguments are given, report an error message to `stderr` and exit the program with `EXIT_FAILURE`. In general, if an error is encountered, display a meaningful error message on `stderr` by using either `perror()` or `fprintf()`, then exit the program with `EXIT_FAILURE`. Only use `perror()` if the given library or system call sets the global `errno` variable.

Error messages must be one line only and use the following format:

```
ERROR: <error-text-here>
```

Submittity may check for the existence of the line above by verifying that the line starts with the "ERROR:" string.

Submission instructions

To submit your assignment and perform **final** testing of your code, we will use the auto-grading features of Submittity. Your code will not be manually reviewed.

To help ensure that your program executes properly, consider making use of the `DEBUG_MODE` pre-processor technique shown below. This helps to avoid accidentally displaying extraneous output in Submittity. Here is an example:

```
#ifdef DEBUG_MODE
    printf( "the value of x is %d and y is %f\n", x, y );
    printf( "here123\n" );
    printf( "why is my program crashing here?!\n" );
#endif
```

To compile this code in “debug” mode, use the `-D` flag as follows:

```
bash$ gcc -Wall -Werror -g -D DEBUG_MODE hw1.c
```

Submittity will use the following to compile your code:

```
bash$ gcc -Wall -Werror hw1.c
```